

Anwendungssysteme

Anne

TODO

Annotations



Reflections

Annotations

Aufgabe 1: *Annotations Intro*

- Wofür können Annotations in Java (und anderen Programmiersprachen) benutzt werden?
- Metadaten über Code
- Geben **zusätzliche Informationen** über Klassen
- Implementieren **nicht direkt Funktionalität**

```
@Author(name="Jane Doe", date="1970-01-01")
public class HelloWorld {
    // ...
}
```

Eigene Annotation

General syntax:

```
<access modifier> @interface AnnotationName {...}
```

Example:

```
public @interface Deprecated {...}
```

You already know:
class, interface, enum.

```
public @interface Author {  
    String name();  
    String date();  
}
```

- Entweder makieren wir etwas oder wir halten "Metadaten"

Wichtig:

- sieht aus wie Methoden
- sind aber **nur Daten (Metadaten)**

Annotations

- Welche Annotations sind in Java eingebaut?
- @Override -> prüft Methodenüberschreibung
- @SuppressWarnings -> Compiler unterdrückt Warnungen
- @Deprecated -> markiert Methode als veraltet
- -> diese annotations sind hart implementiert -> Compiler kennt sie und hat für sie logic eingebaut -> sie Selbst haben keine logik

Eigene Annotation

Wir wollen default Wert:

```
public @interface Example {  
    String [] value() default "test";  
    String author();  
    String date()  
}
```

```
@ Example(author = "TS", date = "2023")  
public class HelloWorld {  
}
```

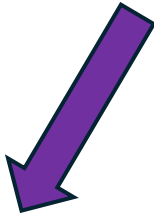
```
@ Example(author = "TS", date =  
"2023", value = {"Katze", "Hund"})  
public class HelloWorld {  
}
```

Supported return types:
primitives, Enums, annotations,
String, Class, arrays of any of these

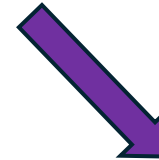
Extra Fakt:

```
public @interface Example {  
    String value()  
  
}
```

- Wenn wir nur ein Element haben -> nenne es value()



```
@ Example("TS")  
public class HelloWorld {  
}
```



```
@ Example(value = "TS")  
public class HelloWorld {  
}
```

- Welche Retentions und Targets können Java Annotations haben?
 - Meta-Annotationen spezifizieren das Verhalten der Annotation selbst:

@Target

worauf darf Annotation angewendet werden

- **ANNOTATION_TYPE**: Andere Annotationen
- **CONSTRUCTOR**: Konstruktor-Deklarationen
- **FIELD**: Variablendeklarationen innerhalb einer Klasse oder eines Objekts
- **LOCAL_VARIABLE**: Lokale Variablendeklarationen innerhalb einer Methode oder Funktion
- **METHOD**: Methodendeklarationen
- **PACKAGE**: Paketdeklarationen
- **PARAMETER**: Parameter von Methoden oder Konstruktoren
- **TYPE**: Typdeklarationen (Klasse, Interface, Enum oder Annotation)
- **TYPE_PARAMETER**: Typ-Parameter-Deklarationen bei Generics (in generischen Klassen, Interfaces, Methoden oder Konstruktoren)

@Target -> wo darf man die Annotation benutzen?

Typ

TYPE

METHOD

FIELD

CONSTRUCTOR

PARAMETER

LOCAL_VARIABLE

Bedeutung

Klassen, Interfaces, Enums

Methoden

Variablen

Konstruktoren

Parameter

lokale Variablen

@Retention

Bestimmt, wie lange die Annotation verfügbar ist:

- SOURCE: Nur im Quellcode gespeichert.
- CLASS: Bis zur Kompilierung.
- RUNTIME: Auch zur Laufzeit per Reflection abrufbar.

@Retention -> Wie lange existiert die Annotation?

Typ	Bedeutung	Beispiel
SOURCE	Nur im Code (verschwindet nach Compile)	wie Kommentare
CLASS	Im Bytecode, aber nicht zur Laufzeit	Standard
RUNTIME	Auch zur Laufzeit verfügbar (Reflection möglich)	wichtig für Frameworks

Beispiel: source

```
@Retention(RetentionPolicy.SOURCE)
```

```
public @interface Todo {  
    String value();  
}
```

```
@Todo("Methode noch fertig implementieren")  
public void calculate(){
```

-> Du siehst Annotation im Code, Compiler kann sie für Warnungen nutzen

@Repeatable

@Repeatable(B.class)

```
public @interface A{  
    String value();  
}
```

```
public @interface B{  
    A[] value();  
}
```

```
@A ("OWNER")  
@A ("ADMIN")  
@A ("USER")  
@A ("GUEST")  
public String role;
```

Zusammenfassung:

“@” to mark Annotation

Interface-like definition without parameters

optional default

```
public @interface ClassMetadata {  
    String author();  
    String date();  
    String[] reviewers() default "None";  
}
```

Supported return types:
primitives, enums, annotations,
String, Class, arrays of any of these

passing values to the elements

```
@ClassMetadata(author = "TS", date = "2023", reviewers = {"DB", "MG"})  
public class HelloWorld {  
    // ...  
}
```

Beispiel: Lombok

- Wollen Standard-Code reduzieren
- @toString
- @Getter
- @Setter
- @Log

```
@ToString  
@AllArgsConstructor  
@Log  
public class User {  
    private @Getter @Setter String name;  
}
```


@Log

Ein Logger fügt automatisch wichtige Metadaten hinzu, die du bei println manuell tippen müsstest:

- **Zeitstempel** (Wann genau ist der Fehler passiert?)
- **Klassenname** (In welcher Datei trat das Problem auf?)

```
Log.severe("...")  
Log.info("Die Methode wurde mit dem  
Namen " + name + " aufgerufen")  
)
```

```
Apr 29, 2026 11:58:10 AM User doSomething  
INFO: Die Methode wurde mit dem Namen Laura aufgerufen  
Apr 29, 2026 11:58:10 AM User doSomething  
SEVERE: Fehler:  
java.lang.ArithmeticException Create breakpoint : / by zero  
    at User.doSomething(User.java:21)  
    at User.main(User.java:54)
```

Aufgabe

Aufgabe 3 – Wiederholte Annotations

Wir bleiben beim Szenario aus der vorherigen Aufgabe. Unsere Anwendung wächst und wir brauchen mittlerweile eine bessere Rollenverwaltung, damit nicht alle User auf Dinge zugreifen können, die nur für uns vorgesehen sind. Erweitern Sie zunächst die User-Klasse aus der vorherigen Aufgabe um ein Attribut `role`, was durch einen String dargestellt werden soll. Implementieren Sie nun eine Annotation `AllowedValue`, welche die möglichen Werte darstellt. Stellen Sie dabei sicher, dass eine Variable mehrere erlaubte Werte haben darf. Die Annotation soll wie folgt verwendet werden.

```
@AllowedValue("OWNER")  
@AllowedValue("ADMIN")  
@AllowedValue("USER")  
@AllowedValue("GUEST")  
public String role;
```

Aufgabe 3: *Reflections Intro*

- Was sind Reflections?
 - Wofür können sie eingesetzt werden?
 - Wann sollte man sie eher nicht benutzen?
-
- ermöglichen Programmen, Informationen über sich selbst zur Laufzeit zu erhalten
 - gegebenenfalls das Verhalten zu ändern
 - Dies schließt das Abrufen von Methoden, Feldern, Konstruktoren und Annotationen mit ein.

Reflections

Nachteile: komplexes debugging, Performanceverlust

- `Class<T>`: repräsentiert Klassen und Interfaces
- `Constructor<T>`: gibt Informationen über Konstruktor
- `Field`: repräsentiert Feld von Klasse oder Interface.
- `Method`: repräsentiert Methode
- `Parameter`: repräsentiert Parameter einer Methode
- `Annotation`: repräsentiert Annotation

Wie bekommt man Class Obejct?

```
Class<Person> c = Person.class;
```

Oder

```
Person p = new Person("Max");  
Class<?> c2 = p.getClass();
```

```
System.out.println(c.getName()); // so bekommen wir Klassennname
```

GitHUb –
Code kommt
später

